

Chapter 5. Effective Transformations

Most commonly, Spark programs are structured on RDDs: they involve reading data from stable storage into the RDD format, performing a number of computations and data transformations on the RDD, and writing the result RDD to stable storage or collecting to the driver. Thus, most of the power of Spark comes from its transformations: operations that are defined on RDDs and return RDDs.

At present, Spark contains specialized functionality for about a half-dozen types of RDDs, each with its own properties and scores of different transformation functions. In this section, we hope to give you the tools to think about how your RDD transformation, or series of transformations, will be evaluated. In particular: what kinds of RDDs these transformations return, whether persisting or checkpointing RDDs between transformations will make your computation more efficient, and how a given series of transformations could be executed in the most performant way possible.

NOTE

The transformations in this section are those associated with the RDD object used in Spark Core (and MLlib). RDDs are also used inside of DStreams with Spark Streaming, but they have different functionality and performance properties. Likewise, most of the functions discussed in this chapter are not yet supported in `DataFrames`. Since Spark SQL has a different optimizer, not all of the conceptual lessons of this chapter will carry over to the Spark SQL world.

TIP

As Spark moves forward, more RDD transformations will become available on `Datasets`, which can be used in Spark SQL, and which are discussed in [“Datasets”](#).

Narrow Versus Wide Transformations

In [Chapter 2](#), we introduced one important distinction between types of transformations: those with *wide dependencies* and those with *narrow dependencies*. This distinction is important because it has strong implications for how transformations are evaluated and, consequently, for their performance. In this subsection, we will more precisely define the wide and narrow transformations, demonstrate how to determine whether a transformation is wide or narrow, and explain why this distinction matters for evaluation and performance.

TIP

Recall that Spark is lazily evaluated, meaning that a transformation is not executed until an action that depends on that transformation is called. This, as we discussed in detail in [“Lazy Evaluation”](#), has important consequences for fault tolerance, performance, and debugging. If the information in this tip is confusing, please refer back to [Chapter 2](#), which will give you the basic understanding of the Spark execution engine needed for this chapter.

To summarize what we covered in [Chapter 2](#): wide transformations are those that require a shuffle, while narrow transformations are those that do not. In [“Wide Versus Narrow Dependencies”](#) we explained that in narrow transformations, the child partitions (the partitions in the resulting RDD)

depend on a known subset of the parent partitions. While this definition is correct, it is less precise than the formal definition of narrow transformations.

The 2012 paper that first presented the evaluation semantics for Spark defines transformations with narrow dependencies as those in which “each partition of the parent RDD is used by at most one partition of the child RDD.” The creators define transformations with wide dependencies as transformations in which “multiple child partitions may depend on [each partition in the parent].” This definition states the analogue of what we explained in Chapter 2, in which we defined narrow and wide dependencies in relation to the child RDD’s dependencies. In contrast, the creators’ definition defined narrow and wide dependencies in terms of the dependencies on the parent RDD, rather than those on the child RDD.

We think the definition presented in Chapter 2 is easier to conceptualize since one usually designs a program by thinking from the input data (parent RDD) to the output data (child RDD). However, the Spark evaluation engine (the “DAG”) builds an execution plan in reverse: from the output (the last action) to the input RDD. Thus, the Spark creators’ definition mirrors the way that Spark is evaluated and consequently it is more precise in two important ways. First, the founders’ definition rules out the case of one parent partition having multiple children in a narrow dependency. It explains why `coalesce` is only a narrow transformation when it is reducing rather than increasing the number of partitions. Second, the founders’ definition clarifies why the number of tasks used to complete a computation corresponds to each *output partition* rather than each *input partition*—when RDDs are evaluated; the tasks needed to compute a transformation are computed on the child partitions.

Figure 5-1 shows dependencies between parent and child partitions for narrow and wide transformations for the Spark program in [Example 5-1](#). Assume RDD1 is an RDD of integers.

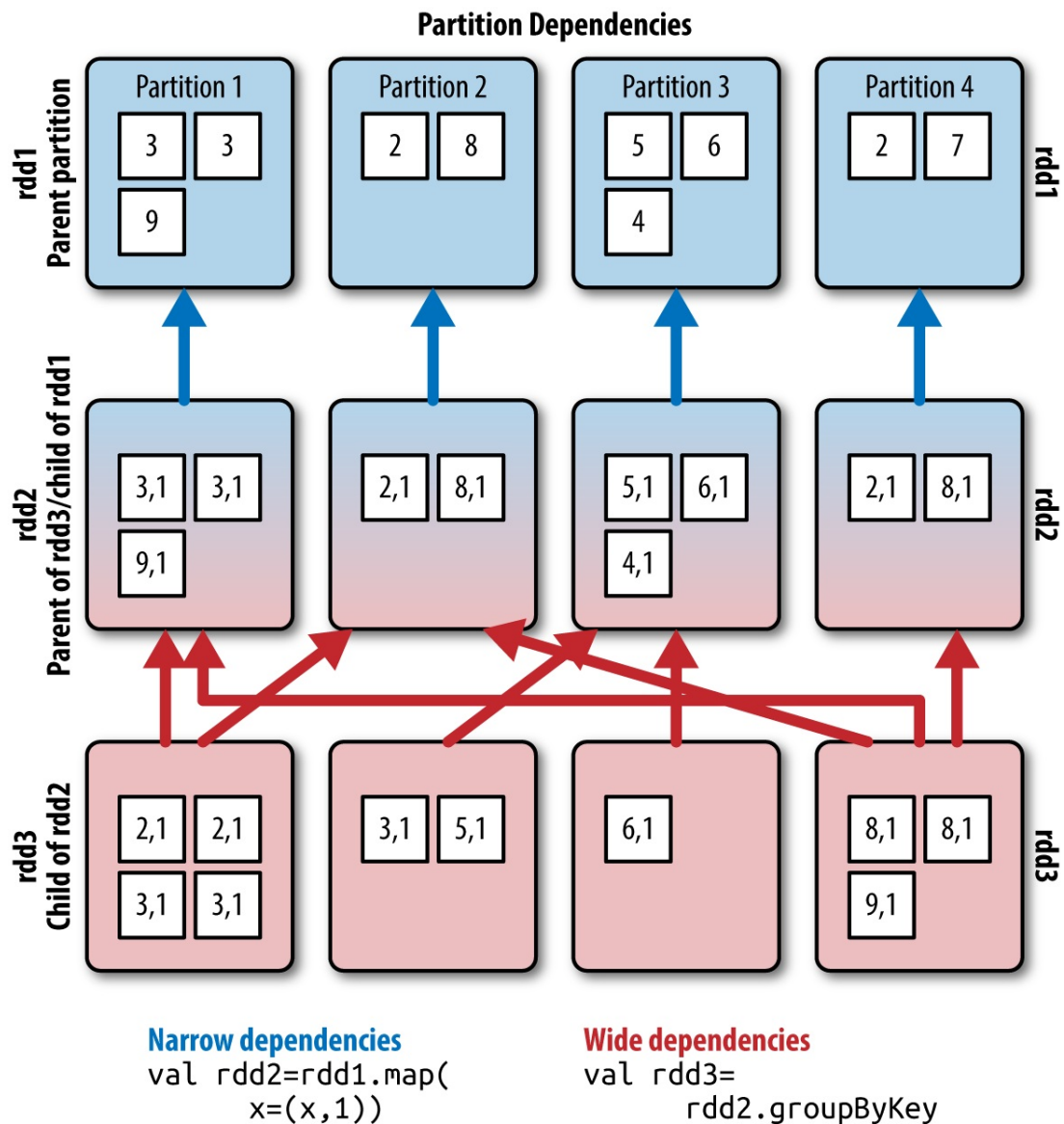


Figure 5-1. Narrow versus wide dependencies between partitions

Example 5-1. Narrow versus wide example

```
//Narrow dependency. Map the rdd to tuples of (x, 1)
val rdd2 = rdd1.map(x => (x, 1))
//wide dependency groupByKey
val rdd3 = rdd2.groupByKey()
```

We use the same structure as the diagrams presented in Figures [2-2](#) and [2-3](#). Arrows represent partition dependencies. Each child partition has arrows pointing to the parent partitions upon which it depends; if an arrow points from partition y to partition x, that means that x depends on y. Blue arrows represent narrow dependencies and red arrows represent wide dependencies.

We assume the RDD has four partitions. Unlike the diagrams presented in [Chapter 2](#), here we will show how actual records in a very small RDD might be distributed amongst the partitions. In this case we show how RDD1, RDD2, and RDD3 would be partitioned if RDD1 was an RDD of the integers 3, 3, 9, 2, 8, 5, 6, 7.

As you can see, to compute the `map` step, each child partition depends on just one parent, since the data doesn't need to be moved between partitions for the operation to be computed. However, in the `groupByKey` step, Spark needs to move an arbitrary number of the records so that those with the same key are in the same partition in order to combine records corresponding to a single key into one iterator (recall that iterator is a local, rather than a distributed collection). Thus, the child partitions depend on many partitions in the parent RDD.

NOTE

This diagram is intended to show how partitions, an abstract concept used in Spark evaluation, depend on each other, rather than any physical data movement across machines. Each line of squares in the diagram represents the same executors at different points in time. The arrows denote dependencies between partitions. In fact repartitioning data does not necessarily require data movement across machines, since partitions may reside on the same executor. When changing the partition of a record does require data movement between executors, the records have to be passed through the driver rather than transferred directly between the executors.

Implications for Performance

In [“Wide Versus Narrow Dependencies”](#) we asserted that transformations with narrow dependencies are faster to execute partly because narrow transformations can be combined and executed in one pass of the data. In this section we hope to explain why this is from an evaluation perspective.

Narrow dependencies do not require data to be moved across partitions. Consequently narrow transformations don’t require communication with the driver node, and an arbitrary number of narrow transformations can be executed on any subset of the records (any partition) given one set of instructions from the driver. In Spark terminology, we say that each series of narrow transformations can be computed in the same “stage” of the query execution plan.

In contrast, as we stated in [“The Anatomy of a Spark Job”](#), a shuffle associated with a wide dependency marks a new stage in the RDD’s evaluation. Because tasks must be computed on a single partition and the data needed to compute each partition of a wide dependency may be spread across machines, transformations with wide dependencies may require data to be moved across partitions. Thus, the downstream computations cannot be computed before the shuffle finishes.

For example, it should be intuitive that sorting cannot be accomplished with narrow transformations because sorting requires an order to be defined on all of the records—not just within each partition. Indeed, the `sortByKey` function has wide dependencies. It requires the data to be partitioned, so all the keys within a certain range live on the same partition. That way, sorting the data on each partition leads to a sorted

result. Any narrow transformations following the sort cannot be done until after the shuffle completes because the records on each partition may change.

Stage boundaries have important performance consequences. Except in the case of multiple RDD operations like `join`, the stages associated with one RDD must be executed in sequence (see [Chapter 4](#)). Thus, not only are shuffles expensive since they require data movement and potential disk I/O (for the shuffle files), they also limit parallelization.

Implications for Fault Tolerance

The cost of failure for a partition with wide dependencies is much higher than for one with narrow dependencies, since it requires more partitions to be recomputed. If one partition in the parent of a `mappedRDD` (the resulting RDD type of a `map` operations) fails, for example, only one of its children must be recomputed, and the tasks needed to recompute that child partition can be distributed across the executors to make this recomputation faster. In contrast, if the parent of the sorted RDD loses a partition, it is possible (in the worst case) that all the child partitions will need to be recomputed. For this reason, the cost of recomputing a partition in the case of failure for a partition with wide dependencies is much higher than for a partition with narrow dependencies.

Chaining together transformations with wide dependencies only increases the risk of a very expensive recomputation particularly if any of the wide transformations have a high probability of causing memory errors. In some instances, the cost of recomputation may be high enough that it is worth `checkpointing` an RDD, so the intermediate results are saved. We will discuss checkpointing in detail in [“Reusing RDDs”](#).

The Special Case of coalesce

The `coalesce` operation is used to change the number of partitions in an RDD. As shown by the diagram in [Figure 2-2 in Chapter 2](#), when `coalesce` reduces the number of output partitions, each parent partition is used in exactly one child partition since the child partitions are the union of several parents. Thus, according to our definition of narrow dependencies, `coalesce` is a narrow transformation even though it changes the number of partitions in the RDD. Since tasks are executed on the child partition, the number of tasks executed in a stage that includes a `coalesce` operation is equivalent to the number of partitions in the result RDD of the `coalesce` transformation.

TIP

Using `coalesce`, the number of partitions can decrease in one stage without causing a shuffle. However, `coalesce` causes the upstream partitions in the entire stage to execute with the level of parallelism assigned by `coalesce`, which may be undesirable in some cases. Avoid this behavior at the cost of a shuffle by setting the shuffle argument of `coalesce` to `true` or by using the `repartition` function instead.

However, when `coalesce` increases the number of partitions, each parent partition necessarily depends on several child partitions. Thus, according to the more precise definition of wide dependencies presented in [“Narrow Versus Wide Transformations”](#), using `coalesce` to increase the number of partitions is a wide transformation. The `coalesce` function prioritizes evenly distributing the data across the child partitions. Consequently, the location of records in the output cannot be determined at design time because it depends on how many records are stored on each

input partition, and the number of records on each partition of course cannot be determined without reading the data and evaluating the upstream transformations. Ergo, increasing the number of partitions with either a `coalesce` or `repartition` call requires a shuffle.

What Type of RDD Does Your Transformation Return?

RDDs are an abstracted concept in two ways: they can be of almost any arbitrary type of record (e.g., `String`, `Row`, `Tuple`), and they can also be members of one of several implementations of the RDD interface with varying properties. Both distinctions are important for performance and evaluation. The first is important, because some transformations can only be applied to RDDs with certain record types. The second is important because each transformation returns one of the several implementations of the RDD interface, and therefore the same transformation called on two different RDD implementations (such as a `mappedRDD` versus a `GoGroupedRDD`) may be evaluated differently. In particular, some RDD implementations retain information about the ordering or locality of the records in the RDD from previous transformations. Understanding the data locality and partitioning information associated with the resulting RDD of a transformation can help avoid unnecessary shuffles. We will save a more detailed discussion of this for “Preserving Partitioning Information Across Transformations” since it is most relevant to Pair RDDs. In this section we will discuss preserving record type information because it can be important for performance and surprisingly difficult as Spark programs get complicated.

The RDD is a collection type which, much like collection types in Scala,

Java, and most other strongly typed languages, is instantiated with a type parameter indicating the type of the members in the collection.

NOTE

In Scala, the syntax for a type parameter is brackets; e.g., `List[String]`, which indicates a sequence of `String` objects. It is equivalent to the Java syntax `< >` (e.g., `List<String>`).

RDDs are similarly typed. For example, if you use `sc.textfile` to read in your RDD, you will end up with an RDD of `String` type (denoted `RDD[String]` in Scala and `RDD<String>` in Java).

The RDD's record type information is important because many transformations are only defined on RDDs that are of a particular type, so trying to use methods on an RDD of generic type will return compile-time or runtime errors. For example, if an RDD of tuples has lost its type information and is interpreted by the compiler to be of type `RDD[Any]` or even `RDD[(Any, Int)]`, calling `sortByKey` will not compile. The compilation error occurs because `sortByKey` can only be called on RDDs of key/value pairs where the keys have some implicit ordering. Similarly, numeric functions such as `max`, `min`, and `sum` can only be called on RDDs of `Long`, `Int`, or `Double`.

Record type information is one of many places in the Spark API where implicit conversions are likely to cause difficulties. If you are writing subroutines to be used in RDD transformation, it is often best to specify the helper function's input and return types concretely and avoid writing them on a generic type.

One instance that often leads to problems losing type information is when working with `DataFrames` as `RDDs`. `DataFrames` can be implicitly converted to `RDDs` of `Rows`. However, since the Spark SQL `Row` object is not strongly typed (it can be created from sequences of any value type), the Scala compiler cannot “remember” the type of value used to create the row. Indexing a row will return a value of type `Any`, which must be cast to a more specific type, such as a `String` or an `Int`, to perform most calculations. The type information for the rows is stored in the schema. However, converting to an `RDD` throws away a `DataFrame`’s schema information, so it is important to bind the `DataFrame` schema to a variable. One of the advantages of the `Dataset` API is that it is strongly typed, so the values in each row will retain their type information even after conversion to an `RDD`.

Minimizing Object Creation

“Garbage collection” is the process of freeing up the memory allocated for an object once that object is no longer needed. Since Spark runs in the JVM, which has automatic memory management and large data structures, garbage collection can quickly become an expensive part of our Spark job. Garbage collection or “GC” errors are a common cause of failure. Even if garbage collection overhead doesn’t prohibit a job from running, garbage collection creates additional serialization time, which can significantly slow it down. We can minimize the GC cost by reducing the number of objects and the size of those objects. We can reduce the size and number of our objects by reusing existing objects and by using data structures (such as primitive types) that take up less space in memory.

Reusing Existing Objects